BIRLASOFT FORGE FDE ACADEMY · SPRINT 3

The Template Is Not The Cloud

Day 43 — Azure Bicep: infrastructure as code, module by module
Day 44 — The full DecisionStream stack, deployed and priced

30 September 2026 · 4 hours · Infrastructure Hardening & DevOps

Confidential · For Learner Use Only

Where you are, and what these two days add

Days	What you did	What it left you with
41–42	Least privilege, killed the keys, hardened secrets	A secure system — that exists only in one subscription
43	Describe every piece of it in Bicep	A system somebody else could rebuild
44	Deploy the whole thing from that description, and price it	A number you can put in front of a client
45–52	CI/CD, observability, hand-over	A system the client's team can own

WHY THIS COMES AFTER THE SECURITY WORK, NOT BEFORE

If you write the Bicep first, you write down today's mistakes and make them permanent — every future environment gets the same over-broad roles and the same live access keys. Days 41 and 42 decided what the correct infrastructure looks like. Today we make that correct version repeatable. Order matters.

A REAL SCENE

The client asks for a UAT environment, identical to production, by Friday. If your infrastructure is in Bicep, that is one command and a parameter file. If it is not, it is three days of clicking, and the UAT environment will be subtly different from production in ways nobody discovers until go-live weekend.

DAY 43 · THE PROVOCATION

One question. Answer it honestly before we start

“Your resource group is deleted by accident tonight. How long until DecisionStream is serving claims again — and what would be missing?”

- **The three answers you will hear:**
  - “A few hours, we have the Bicep.” Then when did you last run it from empty?
  - “A week, maybe.” Honest, and most common.
  - “I do not know what all is in there.” That is the finding, and it is normal.

THE SECOND HALF IS THE REAL QUESTION

Not how long — what would be MISSING. The API Management policy somebody edited in the portal. The firewall rule added during an incident. The app setting that was never written down. Those are the pieces that will not come back, and nobody has a list of them.

WHERE WE LAND BY 8:30PM

“One command, about forty minutes, and the only thing not restored is data — which is a different control. I know that because we rebuilt it from empty into a throwaway resource group on Tuesday, and here is the test output.” Notice the second sentence. The claim is worth nothing without it; having run it is the whole deliverable.

THE ARGUMENT FOR BOTH DAYS

Your Bicep file says what you meant. The cloud says what actually happened.

Four separate mechanisms let the two drift apart. Knowing all four is the job.

1

Incremental mode never deletes

Remove a resource from your Bicep and it stays in the cloud, forever.

2

People change things by hand

And the portal does not ask them to update your repository.

3

What-if cannot see everything

Keys, references and secure values are invisible to it, by design.

4

Some settings are not in Bicep at all

Data, tokens, usage — and those are what the bill is made of.

Bicep in five minutes, for everybody in the room

Bicep is a simpler language that compiles into ARM JSON. Azure only ever understands ARM. Bicep exists because ARM JSON is unpleasant to write and impossible to review.

THE WHOLE LANGUAGE, ROUGHLY

```
param location string = resourceGroup().location
param env string
var accountName = 'cosmos-ds-${env}'

resource cosmos 'Microsoft.DocumentDB/
        databaseAccounts@2024-05-15' = {
  name: accountName
  location: location
  properties: { disableLocalAuth: true }
}
output cosmosId string = cosmos.id
```

`param`	What changes between environments. Passed in from outside.
`var`	A value you compute once and reuse. Never passed in.
`resource`	One thing in Azure. The version after @ is the API version and it matters.
`module`	A Bicep file called by another Bicep file. This is how you stop writing one huge file.
`output`	A value handed back to whoever called you. Safe for IDs. Never for secrets.

TWO THINGS THAT TRIP PEOPLE UP ON DAY ONE

The API version after the @ sign is part of the resource type, not decoration. Change it and properties appear or disappear — half of all “this worked yesterday” Bicep problems are an API version difference.

And Bicep is declarative, not a script. You are not telling Azure what to do, you are describing what should exist. Azure works out the difference. That is why there is no “if already exists” — the whole language assumes you will run it again and again.

DAY 43 · INTENT

Why you, the FDE, own the infrastructure code

Because you are the only one who knows the shape

The client's platform team knows Azure. They do not know that your extraction Function needs a VNet route to Document Intelligence and nothing else. The connection graph you drew on Day 41 is the input to the Bicep, and only you have it.

Because hand-over is the whole engagement

An FDE engagement ends with somebody else owning the system. If the environment can only be recreated by you, from memory, you have not delivered a system — you have delivered a dependency on yourself.

Because the price conversation needs a number

“It will cost roughly what the pilot cost” ends renewal conversations badly. A cost estimate produced from the infrastructure definition, with its exclusions named, is a professional artefact. We build one tomorrow.

DAY 43 · METHOD

Three modules, and why the boundaries are where they are

The brief asks for three stacks. The interesting question is not how to write a module — it is where to draw the line between them. Draw it wrong and every deployment touches everything.

INFERENCE STACK

VNet · 6 Functions · API Management · App Service plan

Changes weekly. Small blast radius wanted.

FRAUD STACK

Cosmos DB · model endpoint · Blob · Key Vault

Changes rarely. Holds the data. Must not be casually redeployed.

OBSERVABILITY STACK

Log Analytics · App Insights · workbooks · alerts

Deployed first, torn down last. Everything else depends on it.

THE RULE FOR DRAWING THE LINE

Group by lifecycle, not by technology. Things that change together belong together.

THE FAILURE THIS PREVENTS

One giant module means every small change redeploys the database. Eventually someone stops deploying.

Ask of every module: if this deployment fails halfway, what is broken? If the answer includes “the claims database” for a change to a Function, the boundary is in the wrong place.

DAY 43 · METHOD

Parameters, variables, outputs — and the one hard rule

	What it is for	Decorate it with	Watch out for
param	Anything that differs per environment	@description @allowed @secure	Too many params is a design smell
var	A value computed once inside the file	—	Naming that hides what it builds
output	Handing an ID back to the caller or pipeline	@description	Outputs are stored in plain text
module	Calling one Bicep file from another	—	Passing whole objects around

NEVER PUT A SECRET IN AN OUTPUT. THIS IS NOT A STYLE PREFERENCE.

Deployment outputs are saved in the resource group's deployment history, in plain text, and anyone with Reader on the resource group can read them — months later. An output containing a key or a connection string is a credential you have published to everybody who can see the subscription, and rotating the key does not remove the old value from the history.

SAFE TO OUTPUT

Resource IDs · endpoint URLs · Key Vault secret URIs (the address of a secret is not the secret) · principal IDs of managed identities

NEVER OUTPUT

Keys · connection strings · passwords · anything from listKeys() · anything you would not paste into a group chat

DAY 43 · METHOD

Dev, staging, prod — parameter files, not if-statements

The wrong way is one Bicep file full of conditionals on the environment name. It reads badly, tests badly, and the production path is the one least often exercised.

prod.bicepparam

```
using './main.bicep'

param env           = 'prod'
param location      = 'centralindia'
param cosmosTier    = 'Standard'
param minReplicas   = 2
param enablePublicNetwork = false
```

- **One template, three parameter files**
  - The shape of every environment is identical
  - The differences are visible in one short file
  - A reviewer can diff prod against dev in seconds
  -
- **Keep the differences boring**
  - Size, count, tier, region, network exposure
  - If dev and prod differ in SHAPE, dev is not a test

THE DIFFERENCE THAT ACTUALLY BITES

Dev uses a Consumption plan, prod uses Premium. They behave differently on cold start, on concurrency and on VNet integration. Every bug you cannot reproduce in dev traces back to a shape difference like this one. Name them all explicitly in a table in your ADR, so nobody is surprised.

A good test of your parameter files: hand prod.bicepparam to somebody who has never seen the project, and ask them what is different about production. If they can answer in a minute, you have done this well.

Bicep does not have “aspects” — it has four other layers

WHERE THE TERM COMES FROM

“Aspects” is a concept from AWS CDK, where you can attach a visitor that walks the resource tree and mutates or validates every node. Bicep has no equivalent feature. What Bicep has instead is four separate guardrail layers, each firing at a different moment — and knowing which one to reach for is the actual skill.

Layer	Fires when	What it is good at	What it cannot do
Bicep linter (bicepconfig.json)	As you type	Style, unused params, insecure defaults	Know anything about your policies
PSRule for Azure	In the build, on a PR	Well-Architected rules against the template	See the running environment
Azure Policy	At deployment, in Azure	Actually blocking a non-compliant deploy	Help you before you push
Deployment stacks (denySettings)	After deployment	Stopping people editing what you deployed	Prevent the first mistake

THE ONE LINE TO REMEMBER

The linter and PSRule tell you about the template. Azure Policy and deny settings tell you about the cloud. You need both, because the template is not the cloud — and a team that only runs PSRule will pass every check while somebody edits the live resource by hand.

DAY 43 · METHOD

PSRule for Azure — useful, and not a security review

PSRule reads your compiled template and checks it against several hundred Azure Well-Architected rules. It is genuinely good, it is free, and it belongs in your pipeline from day one.

- **What it reliably catches**
  - Storage or Cosmos with public network access left open
  - Missing diagnostic settings, so nothing gets logged
  - TLS below 1.2, HTTPS not enforced
  - Key-based auth still enabled — your Day 42 work, checked
  - No managed identity where one is available
  - Missing tags your baseline requires

WHAT IT CANNOT CATCH

That you gave the Function Contributor instead of a data role. That your module deploys to the wrong subscription. That the resource exists in Azure but not in your template. PSRule reads a file — it has no idea what is actually running.

```
az bicep build --file main.bicep --outfile main.json
Assert-PSRule -InputPath main.json -Module PSRule.Rules.Azure -Outcome Fail    # fail the PR on any Fail
```

DECIDE YOUR BASELINE BEFORE THE FIRST RUN, NOT AFTER

The default rule set will produce a long list on any real project. Agree which rules are blocking and which are advisory up front — otherwise the team simply lowers the bar until the pipeline goes green, which is the same failure as redefining “critical” on Day 41.

You cannot unit test the cloud — so test what you can

WHAT A "BICEP UNIT TEST" ACTUALLY IS

The brief asks for Bicep unit tests in pytest. That is a real and useful practice, but be precise about what it proves: it compiles your Bicep to ARM and asserts on the JSON you produced. It tells you the template you generated is the template you meant. It says nothing at all about whether Azure will accept it, or what you will get.

Level	What it does	Catches	Cost to run
Lint	Read the Bicep	Style, unused params, bad defaults	Instant
Compile + assert	Build ARM, assert on JSON	Wrong SKU, missing property, bad wiring	Seconds
What-if	Ask Azure to predict	Unexpected deletes, surprise replacements	~1 minute
Ephemeral deploy	Deploy to a throwaway RG	Everything the others miss	Minutes, and money
Smoke test	Call the deployed system	That it actually works	Minutes

Only the bottom two rows test the cloud. The top three test your file. Both are worth doing — but only one of them answers the question from slide 3.

What-if: a forecast, and here is what it cannot see

What-if asks Azure to predict the changes a deployment would make. It is the best preview available and you should always run it. It is also wrong in specific, documented, predictable ways.

What-if cannot evaluate	So what you see	Why it matters here
listKeys() and similar	A change, every single time	Every key or connection string line
reference() to other resources	A change, every single time	Cross-module wiring always looks dirty
Secure parameters	Never shown	Your secrets path is invisible to review
utcNow(), newGuid()	The literal function text	Any tagged-with-deploy-time resource
templateLink in nested deploys	Resources simply missing	Template Specs vanish from the preview
Past 500 modules / 5 minutes	Change type becomes “Ignore”	Big estates silently stop being checked

MICROSOFT'S OWN WORD FOR THIS IS “NOISE”

The documentation states plainly that properties can be reported as deleted when they will not change — because they are absent from your Bicep but set to a default during deployment. So a what-if output full of alarming red deletions is the normal, expected, healthy case. That is exactly what makes it dangerous.

The noise trap — and how to use what-if properly

A tool that cries wolf forty times teaches you not to look on the forty-first.

- **How the accident happens**
  - Week 1: the team reads every line of what-if output
  - Week 3: they have learned most red lines are noise
  - Week 6: somebody skims, sees the usual red, and approves
  - Week 6: that one was a real delete of a Cosmos container

HOW TO USE IT SO IT STAYS USEFUL

Do not read what-if as prose. Filter it. Fail the pipeline automatically on any Delete or any resource-replacing change, and let everything else pass without a human reading it. A machine does not get tired of noise.

```
az deployment group what-if -g $RG -f main.bicep -p prod.bicepparam --no-pretty-print > wi.json
python check_whatif.py wi.json --fail-on Delete --fail-on Deploy   # a human only looks when this fails
```

And review what-if against production before the deployment, not during it. An 11pm preview nobody has time to read is theatre.

DAY 44 · INTENT

From three modules to one running system

Yesterday you described pieces. Today you deploy all of it into an empty resource group, get the outputs, and put a monthly number on it. This is the day the claim on slide 3 becomes testable.

1	**Deploy into an EMPTY resource group**	Not your working one. An empty one. Deploying into an environment that already has the resources proves nothing — it will succeed even if half your template is missing.
2	**Collect and verify the outputs**	API endpoint, Cosmos resource ID, Key Vault secret URIs. These are what the pipeline hands to the next stage, so they have to be right and they have to be non-secret.
3	**Run a smoke test against what you built**	One real claim through the deployed system. A green deployment is not a working system; Azure reports success the moment the resources exist.
4	**Produce the monthly cost estimate**	With its exclusions named. That last part is what makes it credible rather than optimistic.

The full stack, and the order it has to go in

Bicep works out most dependencies by itself, from the references between resources. The ones it cannot work out are the ones that will fail your deployment at minute thirty-eight.

Order	Component	Depends on	Typical deploy time
1	Log Analytics + App Insights	Nothing	1–2 min
2	VNet + subnets	Nothing	1–2 min
3	Key Vault	VNet (private endpoint)	2–3 min
4	Cosmos DB, Blob, AI Search	VNet, Key Vault (if CMK)	5–10 min (Cosmos is slow)
5	App Service plan + 6 Functions	VNet, Key Vault, App Insights	3–5 min
6	Role assignments (your Day 41 design)	Every identity above	1 min, but eventually consistent
7	API Management	VNet, Functions	30–45 MIN — plan for it

THE HIDDEN DEPENDENCY

Role assignments are eventually consistent. A Function that starts immediately after its role is granted can still get a 403. Add a retry, or a short wait, in your smoke test.

PLAN THE LAB AROUND ROW 7

API Management takes longer to deploy than this class lasts. Use the Consumption tier for the exercise, or deploy it once beforehand and import it as existing.

DAY 44 · PRE-FLIGHT

Four things that will cost you an evening if nobody warns you

Consumption plan Functions cannot join a VNet

The classic Consumption plan (Y1) has no VNet integration. Your brief asks for both a VNet and six Functions, so you need Flex Consumption, Premium or Dedicated — and that changes the cost estimate you produce tomorrow.

Some Cosmos properties cannot be changed in place

Partition key, and the customer-managed key decision from Day 42. Bicep will not warn you kindly — the deployment simply fails, or worse, the tooling proposes a replacement of a database holding real claims.

Global names are globally unique

Storage accounts, Key Vaults and APIM names are taken across all of Azure. A template that works in your subscription fails in the client's because somebody else owns the name. Use uniqueString(resourceGroup().id) in the name.

Soft delete blocks you from reusing a name

Delete a Key Vault and its name is reserved through the retention window. Your “clean redeploy” then fails with a confusing conflict error. Purge it explicitly, or name the throwaway vault differently each run.

“Provisioned by code” is a claim you have to keep proving

THE SENTENCE IN THE BRIEF, TESTED

The outcome reads “all services provisioned by code”. Here is the problem: the default deployment mode is Incremental, and incremental never deletes. Remove a resource from your Bicep, redeploy, and the resource is still there — running, costing money, and now completely undocumented. Your template and your cloud have quietly stopped matching, and nothing told you.

Option	What happens to a removed resource	When to use it
Incremental (default)	It stays. Forever. Silently.	Everyday deployments — but know the gap
Complete mode	Deleted, along with anything else not in the template	Almost never. One mistake is catastrophic.
Deployment stacks	Managed as a unit; actionOnUnmanage decides delete or detach	The modern answer. Also gives denySettings.

WHAT GOOD LOOKS LIKE

A nightly job that deploys the template into an empty resource group and compares the result against production.

WHAT TO TELL THE CLIENT

“We can guarantee everything in the template exists. We cannot guarantee nothing else does — unless we adopt stacks.”

This is mechanism 1 and mechanism 2 from slide 4. Deployment stacks with denySettings close both at once — which is why they are worth a paragraph in your ADR.

DAY 44 · METHOD

Passing secrets through a deployment, safely

Day 42 removed most of your secrets. The one or two that survived still have to reach the application, and the deployment is where people leak them.

THE SAFE PATTERN

```
resource kv 'Microsoft.KeyVault/vaults@2023-07-01'
        existing = { name: vaultName }
module app './app.bicep' = {
  params: {
    bureauKey: kv.getSecret('bureau-key')
  }
}

// inside app.bicep:
@secure()
param bureauKey string
```

- **Why this one is safe**
  - getSecret() only works on a @secure() param
  - The value never enters the deployment history
  - The value never appears in any log or output
  - It is fetched at deploy time, so rotation just works
  -
- **And the better answer, where you can**
  - Use a Key Vault reference in app settings instead, so the deployment never handles the secret at all

THREE WAYS TEAMS LEAK A SECRET THROUGH A DEPLOYMENT

A plain string parameter instead of @secure() · a secret in an output · a secret in a .bicepparam file committed to git. All three are quiet, and all three survive rotation because the old value stays in history or in the repo.

@secure() hides the value from logs and history. It does NOT encrypt it in your parameter file. A committed prod.bicepparam containing a real secret is still a leak.

DAY 44 · METHOD

The pipeline shape — what runs where

The pipeline is where all of Day 43's checks stop being optional. Its real job is to make the safe path the easy path.

ON EVERY PULL REQUEST

`bicep build · linter · pytest on the compiled ARM · PSRule against the agreed baseline · what-if against production, machine-filtered for Delete and Replace`

Fast, free, no Azure changes. Nothing merges that fails here.

ON MERGE TO MAIN

`Deploy to dev · smoke test · Infracost diff posted as a comment`

If a change raises the monthly bill, the team sees it on the pull request, not on the invoice.

ON RELEASE

`Manual approval · deploy to staging · smoke test · manual approval · deploy to prod`

Two gates, both human. The pipeline should make it easy to do this right and awkward to skip.

DAY 44 · COST

Infracost on Bicep — what it does, and what it needs

Infracost reads a template and prices it, with no credentials and nothing deployed. It does support Bicep, with one step and a few limits worth knowing before you promise a client a number.

IT WILL NOT COMPILE BICEP FOR YOU

```
az bicep build --file main.bicep --outfile out/main.json
infracost breakdown --path out/
infracost diff --path out/ --compare-to base.json
```

- **What it handles well**
  - Fixed monthly costs: plans, tiers, provisioned capacity
  - Copy loops, conditionals and parameter files
  - Diffs between two versions of the template

DOCUMENTED LIMITS

Bicep must be compiled first · @metadata decorators are lost in compilation · resources inside Template Specs are not expanded or priced · public cloud pricing only · subscription and policy-level tags are invisible to it.

A SECOND OPINION IS CHEAP

Azure Cost Estimator (ACE) reads ARM, Bicep and Terraform and uses the Azure Retail Prices API. Running both and comparing is a good habit — where they disagree is usually where your assumptions are.

HOW TO PRESENT THE NUMBER, SO IT SURVIVES CONTACT WITH MONTH THREE

Give three figures, not one: the fixed floor that exists whether or not anybody uses the system; the variable estimate at the volume the client stated; and the same at three times that volume. Then name every exclusion. A client who is told the shape of the cost will forgive a wrong number. A client given one confident figure will not.

DAY 44 · CORRECTION TO THE BRIEF

The monthly cost of an AI system is not in the Bicep file

THE OUTCOME AS WRITTEN WILL MISLEAD SOMEBODY

The brief asks for a “monthly infrastructure cost” from Infracost. That is a real and useful number. It is also, for a system like DecisionStream, very likely the smaller half of the bill — and a client handed it without qualification will budget from it.

Cost driver	In the Bicep file?	Priced by Infracost?	What actually decides it
App Service plan, APIM tier	Yes	Yes	Your SKU choice
Provisioned Cosmos RUs	Yes	Yes	Your throughput setting
Cosmos serverless / autoscale	Partly	Poorly	Real query volume
Model tokens (input + output)	No	No	Claim volume × prompt size
Function executions	No	No	Traffic
Egress, storage growth, logs	No	No	Time and usage

So the honest deliverable is not one number. It is a floor from the template, plus a usage model you built with the client, plus a written list of what is excluded. Say it in that order.

Three lanes — pick one, finish it, defend it

BUILD

Write it and deploy it

- Three Bicep modules: inference, fraud, observability
- Parameter files for dev and prod
- Deploy into an EMPTY resource group
- Verify every output; check none is a secret
- Run one real claim end to end

SHAPE

Test it and price it

- pytest assertions on the compiled ARM output
- PSRule against an agreed blocking baseline
- what-if, machine-filtered for Delete and Replace
- Infracost breakdown, then the diff
- Build the three-figure cost model with exclusions

GOVERN

Decide it and write it down

- Choose the module boundaries and justify them
- Decide incremental vs deployment stacks
- Write the dev-vs-prod shape difference table
- Draft the pipeline stages and the two approval gates
- Write the ADR covering both days

GOVERN and SHAPE need no Bicep syntax at all, and between them they produce the two artefacts a client actually keeps.

Five questions. Evidence, not intention

1	**Delete the resource group. Talk me through the next hour.**	A command, a duration and a date when you last actually did it.
2	**Which of the four drift mechanisms have you closed?**	Slide 4. Naming the ones you have not closed is a better answer than claiming all four.
3	**Show me a what-if output and tell me what is noise.**	If everything looks like a real change to them, they are not reading it — they are scrolling past it.
4	**What is in your cloud that is not in your Bicep?**	The correct answer is never “nothing” unless they can show how they checked.
5	**What will this cost at three times the volume?**	The three-figure model. A single number here is the wrong answer even if it is accurate.

The words, in plain language

ARM

Azure's deployment engine. It only understands JSON; Bicep compiles into it.

Declarative

You describe the end state; Azure works out the steps. Not a script.

Module

A Bicep file called by another Bicep file, with its own parameters and outputs.

API version

The date after @ in a resource type. It decides which properties exist.

Incremental mode

The default. Adds and updates, but never deletes. This is why drift happens.

Complete mode

Deletes anything in the resource group that is not in your template. Dangerous.

Deployment stack

Manages a set of resources as one unit, and can block portal edits to them.

what-if

A predicted change list. Best available preview, and documented to contain noise.

Drift

The gap between what your template says and what is actually running.

Idempotent

Running it again changes nothing. Every good Bicep deployment is this.

CLOSING · DAYS 43–44

What you can say tonight that you could not say this morning

“The whole system is three Bicep modules, and here is why the boundaries are there.”

Architecture, not syntax.

“We rebuilt it from empty on Tuesday. Forty minutes. Here is the smoke test output.”

The answer to slide 3, with evidence.

“Here are the four ways the template and the cloud can drift, and which ones we closed.”

Including the one you did not.

“The floor is this. At your stated volume it is this. At three times, this. Excluded: these.”

A shape, not a single number.

The FDE difference

Anybody can write Bicep. The tutorials are good and the AI assistants are better.

What a client pays an FDE for is the judgement around it: where the module boundaries go, which drift you accepted, and what the number excludes.

All three are written down, not typed in.

Next: Day 45 — CI/CD and observability. Bring your pipeline draft; it becomes the starting point.